

P3 — src/pinn — la PINN

En una frase: una red neuronal que hace dos cosas a la vez — aproxima la trayectoria $t \mapsto [I, E]$ como una curva suave y derivable, y guarda adentro los pesos de Wilson-Cowan a identificar; la “física” (que se cumpla la EDO) se impone por *autograd* sobre esa curva, y así el residuo empuja los pesos hacia su valor real.

Este documento es el manual del paquete `src/pinn`: qué hace cada archivo y cada función/clase, con firma, entradas/salidas y notas. Para la teoría de qué es una PINN, el sesgo espectral y las *Fourier features*, ver **M5 (PINNs)**; para las ecuaciones canónicas de Wilson-Cowan, la tabla de los 10 parámetros y el offset de reposo k_e, k_i , ver **P0** y las ecuaciones canónicas del SPEC. Para el simulador que genera los datos y la implementación de referencia del *right-hand side* (rhs) que la física replica, ver el módulo del modelo (referido en el código como `model.py`).

Panorama del paquete

```
src/pinn/
├── __init__.py      # expone PINN, las 3 losses y Trainer
├── network.py       # clase PINN: arquitectura t -> [I, E] + pesos identificables
├── losses.py        # data_loss, physics_loss, initial_condition_loss
├── train.py         # TrainConfig + Trainer (UNA trayectoria)
└── multitraj.py     # MultiTrajPINN + MultiTrajectoryTrainer (VARIAS trayectorias)
```

La idea de fondo: en un problema **inverso** no queremos la trayectoria, queremos los **parámetros** ($w_{EE}, w_{EI}, w_{IE}, w_{II}$). Pero para escribir la pérdida física — “que la ecuación se cumpla” — hace falta el estado $[I, E]$ y sus derivadas $\dot{I}, \dot{E}$. La red da el estado suave y las derivadas salen gratis por *autograd*. Los pesos viven adentro de la red como `nn.Parameter` y el mismo optimizador que ajusta la red los ajusta a ellos.

`__init__.py`

Nada de lógica: solo reexporta la API pública del paquete.

- `PINN` (de `network.py`)
- `data_loss, physics_loss, initial_condition_loss` (de `losses.py`)
- `Trainer` (de `train.py`)

Nota: `MultiTrajPINN` y `MultiTrajectoryTrainer` **no** se reexportan acá; se importan directo desde `pinn.multitraj`.

`network.py` — la arquitectura

`inv_softplus(y: float) -> torch.Tensor`

Qué hace: invierte la función *softplus*. Como $\text{softplus}(x) = \log(1 + e^x)$ manda cualquier real a un valor **positivo**, la usamos para forzar que los pesos identificados sean > 0 . Guardamos un valor “crudo” (`raw_w`) sin restricción y el peso real es $\text{softplus}(\text{crudo})$. Para arrancar el crudo desde un *guess* inicial dado hay que invertir: $x = \log(e^y - 1)$.

- **Entrada:** `y` (float) — el valor inicial deseado del peso (ya positivo).
- **Salida:** el tensor crudo correspondiente.
- **Nota:** usa `torch.expm1` (numéricamente estable para `y` chico). Los signos de los pesos ya están explícitos en las ecuaciones ($-w_{EI}, -w_{II}$), por eso alcanza con forzar magnitud positiva.

Clase `PINN(nn.Module)`

Red $t \mapsto [I, E]$, con **cero o más** pesos de Wilson-Cowan entrenables.

Constructor:

```
PINN(
    hidden_dim: int = 64,
    n_layers: int = 4,
    t_min: float = 0.0,
    t_max: float = 600.0,
    identify: tuple[str, ...] = ("wEE", "wEI", "wIE", "wII"),
    w_init: dict[str, float] | None = None,
    n_fourier: int = 128,
    fourier_scale: float = 6.0,
)
```

Argumentos:

- **hidden_dim, n_layers:** tamaño del MLP (ancho y profundidad).
- **t_min, t_max:** rango temporal, para **normalizar** la entrada t a $[-1, 1]$. Clave: si se le mete $t = 0 \dots 600$ crudo a una tanh, se satura. Se guardan como *buffers* (viajan con el modelo pero no se entrenan).
- **identify:** tupla con los nombres de los pesos a identificar. () = ninguno.
 - **identify=()** -> **ninguno**. La red solo ajusta la trayectoria: problema **forward** (Paso 4.0, "chequeo de plomería").
 - **identify=("wEE", "wEI", "wIE", "wII")** -> los 4 pesos: problema **inverso** (Pasos 4.1 y 4.2). Es el valor por defecto.
- **w_init:** dict {nombre: valor_inicial}. Es el **punto de partida** de la búsqueda, NO el valor real. Si es None, arranca todos en 1.0.
- **n_fourier:** cuántas frecuencias de *Fourier features* (0 = desactivar -> MLP clásico).
- **fourier_scale:** dispersión de las frecuencias (más grande = capta oscilaciones más rápidas / picos más agudos).

Atributos de clase: `WEIGHT_NAMES = ("wEE", "wEI", "wIE", "wII")`.

Qué construye el `__init__`:

1. **Buffers t_min, t_max** para la normalización.
2. **Fourier features.** En vez de meter el tiempo "pelado" al MLP, lo codifica en senos y cosenos de muchas frecuencias:

$$\text{features}(t) = [\sin(2\pi B t_n), \cos(2\pi B t_n)]$$

Las frecuencias B se sortean una sola vez ($B \sim \mathcal{N}(0, \text{fourier_scale}^2)$), con Generator de semilla fija = 0, reproducible) y quedan **FIJAS** (buffer, no se entrenan). El MLP aprende sólo cómo combinarlas. Esto rompe el **sesgo espectral**: un MLP con tanh no puede ajustar oscilaciones marcadas a partir de t crudo. Si `n_fourier > 0`, la entrada al MLP tiene $2 * \text{n_fourier}$ features; si es 0, entra el t normalizado directo (1 feature). Ver **M5** para la teoría del sesgo espectral.

3. **El MLP (self.net):** Linear(in_features, hidden_dim) + Tanh, luego `n_layers - 1` bloques Linear(hidden, hidden) + Tanh, y una capa final Linear(hidden, 2) **sin** activación -> salida $[I, E]$.
4. **Pesos a identificar.** Si `identify` no está vacío: arma los crudos con `inv_softplus` de cada `w_init[name]` y los guarda apilados en `self.raw_w = nn.Parameter(...)` -> el optimizador los ajusta. Si `identify=()`: `self.register_parameter("raw_w", None)` (forward puro, sin pesos entrenables).

Métodos:

forward(self, t: torch.Tensor) -> torch.Tensor

- **Entrada:** t de forma $(N, 1)$.
- **Salida:** $(N, 2) = [I, E]$.
- **Qué hace:** normaliza $t \rightarrow [-1, 1]$ con `t_min/t_max`; si hay Fourier features arma `proj = 2\pi \cdot t_n \cdot B` y concatena `[sin(proj), cos(proj)]` (forma $(N, 2 \cdot \text{n_fourier})$); si no, usa t_n directo. Pasa el resultado por `self.net`.

identified_weights(self) -> dict[str, torch.Tensor]

- **Salida:** dict {nombre: tensor} con los pesos identificados en su valor **real** (positivo), aplicando `soft-plus(raw_w)`. **Con gradiente** (para usarlos dentro de la pérdida física). Vacío si `identify=()`.

`weights_dict(self) -> dict[str, float]`

- **Salida:** los mismos pesos pero como **floats**, con `.detach()` (sin gradiente). Versión cómoda para leer y loguear.

losses.py — las tres pérdidas

La pérdida total combina:

$$L = w_{\text{data}} \cdot L_{\text{datos}} + w_{\text{physics}} \cdot L_{\text{física}} + w_{\text{ic}} \cdot L_{\text{inicial}}$$

- L_{datos} : que la red pase por las mediciones $[I, E]$.
- $L_{\text{física}}$: que la red cumpla las ecuaciones de Wilson-Cowan. Acá entran los parámetros (identificados y/o fijos): se arma el residuo de la EDO y se lo manda a cero.
- L_{inicial} : que la red respete la condición inicial $[I_0, E_0]$.

```
flowchart TD
    T["t (datos)"] --> NET["PINN: net(t) -> [I,E]"]
    TC["t_c (colocación)"] --> NET
    NET -->|pred vs medición| LD["L_datos = MSE"]
    NET -->|autograd d/dt| DIDE["dI/dt, dE/dt"]
    NET --> IE["I, E"]
    PARAMS["params: fijos + identificados (softplus raw_w)"] --> RHS["rhs_I, rhs_E (ecuación WC)"]
    IE --> RHS
    DIDE --> RES["residuo = d/dt - rhs"]
    RHS --> RES
    RES --> LF["L_física = MSE(residuo)"]
    NET -->|en t0 vs [I0,E0]| LIC["L_inicial"]
    LD --> LTOT["L = w_data·L_datos + w_phys·L_física + w_ic·L_inicial"]
    LF --> LTOT
    LIC --> LTOT
    LTOT -->|backward| OPT["Adam: ajusta red + raw_w"]
```

`data_loss(pred, target) -> torch.Tensor`

- **Entradas:** `pred, target` de forma $(N, 2) = [I, E]$.
- **Salida:** escalar = error cuadrático medio $\langle (pred - target)^2 \rangle$.
- **Qué hace:** MSE simple entre predicción de la red y observaciones.

`physics_loss(model, t_c, P_c, Q_c, params) -> torch.Tensor`

El corazón de la PINN: el residuo de las ecuaciones de Wilson-Cowan en los puntos de colocación.

- **Entradas:**
 - `model`: la PINN.
 - `t_c`: $(M, 1)$ tiempos de colocación donde se exige la EDO.
 - `P_c, Q_c`: $(M, 1)$ los estímulos $P(t) \rightarrow E$ y $Q(t) \rightarrow I$ en esos tiempos.
 - `params`: dict con **todos** los parámetros necesarios: `te, ti, ae, ai, thetae, thetai, ke, ki, wEE, wEI, wIE, wII`. Cada valor puede ser un float (parámetro fijo) o un tensor (peso identificado, con gradiente); el residuo funciona igual en ambos casos.
- **Salida:** escalar = $\langle res_I^2 + res_E^2 \rangle$.
- **Qué hace, paso a paso:**
 1. `t_c = t_c.clone().requires_grad_(True)` — hace falta derivar la salida respecto de t , así que t_c registra gradiente.
 2. `out = model(t_c)`; separa $I = out[:, 0:1]$, $E = out[:, 1:2]$.
 3. **Derivadas por autograd:** `dI = grad(I, t_c, ones_like(I), create_graph=True)[0]` (ídem `dE`). El `create_graph=True` es clave: la pérdida física depende de estas derivadas y tiene que poder backpropagarse a través de ellas.
 4. Arma las entradas a cada población, con el umbral **adentro** de la entrada (misma estructura que `rhs()` en `model.py`):

$$u_i = w_{IE}E - w_{II}I + Q_c - \theta_i, \quad u_e = w_{EE}E - w_{EI}I + P_c - \theta_e$$

5. Right-hand side con la sigmoide `torch.sigmoid(a·u)` (equivale a `expit(a·u)`) y **restando** el offset de reposo k_i, k_e (ver **P0** para por qué: hace que $E = I = 0$ sea equilibrio sin estímulo):

$$\dot{I} = \frac{1}{\tau_i}(-I + S_i(u_i) - k_i), \quad \dot{E} = \frac{1}{\tau_e}(-E + S_e(u_e) - k_e)$$

6. Residuo = (derivada de la red) – (lo que dice la ecuación): `res_I = dI - rhs_I`, `res_E = dE - rhs_E`. Ideal ≈ 0 .

- **Nota:** que `params` acepte tanto float como tensor es lo que permite que el mismo código sirva para forward (todo fijo) y para inverso (los identificados son tensores con gradiente que fluyen hasta `raw_w`).

`initial_condition_loss(model, t0, ic) -> torch.Tensor`

- **Entradas:** `model`, `t0` (1, 1) tiempo inicial, `ic` (1, 2) condición inicial $[I_0, E_0]$.
- **Salida:** escalar = MSE entre `model(t0)` y `ic`.
- **Qué hace:** penaliza que la red se desvíe de la condición inicial.

train.py — entrenamiento de UNA trayectoria

Ajusta, con **un solo optimizador** y a la vez, los pesos internos de la red (para reproducir la trayectoria) y los pesos de Wilson-Cowan que se estén identificando.

Validación temporal (extrapolación): se entrena con **datos** solo hasta cierto tiempo (los primeros 1 — `val_fraction` de la trayectoria) y se evalúa cómo predice el **tramo final** que no vio. La **física (colocación)** **sí se aplica en todo el dominio**: es justamente lo que permite “propagar” la solución hacia el tramo sin datos. Si predice bien ahí, aprendió la dinámica en vez de memorizar puntos.

`@dataclass TrainConfig`

Hiperparámetros de entrenamiento. Campos:

Campo	Default	Qué es
<code>epochs</code>	30 000	tope máximo (la parada por meseta suele cortar antes)
<code>lr</code>	1e-3	learning rate de la red
<code>lr_weights</code>	1e-3	lr de los pesos θ a identificar. Conviene más alto que el de la red: θ arranca lejos y tiene que viajar mucho (ej. 1.0 $\rightarrow$ 6.4)
<code>weight_decay</code>	0.0	regularización L2 de la red (clave bajo ruido: evita que el MLP ajuste el ruido)
<code>w_data</code>	1.0	peso del término de datos
<code>w_physics</code>	1.0	peso del término físico (el “lambda”)
<code>w_ic</code>	1.0	peso de la condición inicial
<code>batch_size</code>	8 000	puntos de DATOS por paso (minibatch)
<code>n_collocation</code>	4 000	puntos de FÍSICA por paso
<code>val_fraction</code>	0.2	fracción final reservada para test (no se entrena con sus datos)
<code>device</code>	“cpu”	dispositivo
<code>log_every</code>	200	cada cuántos epochs loguear
<code>checkpoint_dir</code>	“results/models”	dónde guardar checkpoints
<code>physics_warmup</code>	0	epochs iniciales de “datos primero” (física apagada)

Campo	Default	Qué es
freeze_net_after_warmup	False	al terminar el warmup, congelar la red y ajustar solo θ
early_stop	True	parar al amesetar la pérdida
plateau_tol	1e-3	mejora relativa mínima para considerar "todavía baja"
plateau_patience	10	chequeos seguidos sin mejora antes de cortar
lr_factor	0.5	cuánto se reduce el lr cuando se estanca
lr_patience	6	chequeos sin mejora antes de bajar el lr

Curriculum "datos primero" (physics_warmup): clave para el inverso cuando θ arranca lejos. Durante los primeros physics_warmup epochs se entrena **solo con datos** (física apagada) para que la red aprenda la trayectoria verdadera. Recién después se prende la física, ya con la curva correcta: así el residuo empuja los pesos hacia su valor real en vez de trabarse en un mínimo local.

Congelar la red (freeze_net_after_warmup): al terminar el warmup se congela la red y se ajustan **solo** los pesos de Wilson-Cowan. Evita que la red "haga trampa" deformando la trayectoria para satisfacer la física con pesos errados; obliga a que se muevan los pesos.

Clase Trainer

Orquesta el entrenamiento de la PINN sobre **una** trayectoria.

__init__(self, model, config: TrainConfig, fixed_params: dict)

- model: la PINN.
- config: TrainConfig.
- fixed_params: te, ti, ae, ai, thetai, thetai, ke, ki + los pesos **no** identificados.
- **Qué arma:** manda el modelo al device. Crea un solo optimizador **Adam** con **dos grupos**: la red (model.net.parameters()) con su lr y su weight_decay, y — si raw_w is not None — los pesos a identificar con lr_weights (normalmente más alto). Así θ se mueve rápido sin desestabilizar la red. Agrega un ReduceLROnPlateau (factor=lr_factor, patience=lr_patience, min_lr=1e-5) que baja el lr cuando la pérdida suavizada se estanca; min_lr evita que colapse a cero.

_params(self) -> dict Junta self.fixed con self.model.identified_weights() (los identificados como tensores con gradiente). Es lo que se le pasa a physics_loss.

_prepare(self, dataset) -> tuple

- **Entrada:** dataset dict de np.ndarray con claves t, I, E, P, Q.
- **Salida:** t, target, P, Q, t0, ic, train_idx, test_idx.
- **Qué hace:** pasa todo a tensores float32 columna (N, 1); arma target = [I | E] (N, 2); corte **temporal**: train_idx = primeros 1 — val_fraction, test_idx = resto (el tramo final). t0/ic son el primer punto.

train(self, dataset) -> dict El bucle principal. Devuelve un hist con listas de loss, data, physics, ic, val, una lista por cada peso en model.identify, y las claves stop_epoch, warmup.

Lógica por epoch:

1. **Transición fin del warmup** (epoch == warmup, si warmup > 0): resetea el lr de ambos grupos a sus valores de config, **reinstancia** el ReduceLROnPlateau y reinicia la detección de meseta (EMA, best_ema, sin_mejora) — porque la pérdida da un salto al prender la física. Si freeze_net_after_warmup, pone requires_grad_(False) en toda la red.

2. **Física apagada durante el warmup:** $w_f = 0.0$ si $\text{epoch} < \text{warmup}$, si no $w_f = w_{\text{physics}}$.
3. L_{datos} : minibatch del **tramo de entrenamiento** (primeros 80 %), de tamaño batch_size .
4. $L_{\text{física}}$: minibatch de $n_{\text{collocation}}$ puntos de **todo** el dominio (incluye el tramo de test) — esto es lo que permite extrapolar.
5. L_{inicial} en t_0/ic .
6. $L = w_{\text{data}}L_d + w_fL_f + w_{\text{ic}}L_{\text{ic}}$; `backward(); opt.step()`.
7. **Registro** de las pérdidas y de los pesos actuales.
8. **EMA de la pérdida** ($\text{ema} = 0.99 \cdot \text{ema} + 0.01 \cdot l_{\text{total}}$) para ignorar el ruido del minibatch y mirar la tendencia; `scheduler.step(ema)`.
9. Cada `log_every` epochs: calcula `val` (MSE en el tramo de test), loguea, e imprime los pesos actuales.
10. **Detección de meseta** (solo `early_stop` y $\text{epoch} \geq \text{warmup}$): si $\text{ema} < \text{best_ema} \cdot (1 - \text{plateau_tol})$ hubo mejora (resetea `sin_mejora`); si no, incrementa `sin_mejora`, y al llegar a `plateau_patience` corta el entrenamiento (`stop_epoch = epoch`, `break`).

`_val_mse(self, t, target, test_idx) -> float (@torch.no_grad)` MSE en el tramo de test (los datos que la red **no** usó para entrenar). nan si no hay test.

`predict(self, t_array) -> np.ndarray (@torch.no_grad)` Predicción sobre tiempos dados (para graficar). Devuelve numpy (N, 2).

`save_checkpoint(self, path) / load_checkpoint(self, path)` Guarda/carga {`model_state`, `opt_state`, `fixed`}. `save` crea el directorio padre si no existe.

multitraj.py — VARIAS trayectorias (Experimento 2)

Por qué existe. En una **sola** trayectoria, w_{IE} y w_{II} se compensan en la entrada inhibitoria $u_i = w_{IE}E - w_{II}I$ (degeneración) y no se pueden separar — w_{II} es el cuello de botella del proyecto. Con **varias** trayectorias que exciten E e I de formas distintas, esa compensación ya no funciona en todas a la vez -> el par queda determinado. Es la mitigación estructural de la fragilidad de w_{II} (ver **M5** y los diagnósticos Fisher+SVD del proyecto).

Cómo. Una red por trayectoria (cada una mapea $t \rightarrow [I, E]$ de **su** grabación) y un solo θ compartido. Receta heredada del Paso 4.2: 1. **Warmup:** cada red aprende su trayectoria solo con datos (física apagada). 2. **Congelar** todas las redes. 3. Ajustar **solo** θ (con lr propio y alto) minimizando la **suma** de los residuos físicos de todas las trayectorias.

Clase MultiTrajPINN(nn.Module)

N redes (una por trayectoria) + un θ (4 pesos) **compartido**.

`__init__(self, n_traj, t_min, t_max, hidden_dim=64, n_layers=4, n_fourier=128, fourier_scale=6.0, w_init=None)`

- Crea `self.nets = nn.ModuleList(...)` con `n_traj` instancias de PINN, todas con `identify=()` (sin pesos propios: los pesos viven aparte, compartidos). Todas comparten `t_min`, `t_max` y la config de Fourier.
- Crea `self.raw_w = nn.Parameter(...)` con los 4 pesos crudos (`inv_softplus(w_init[k])`), en orden `WEIGHT_NAMES = ("wEE", "wEI", "wIE", "wII")`. `w_init` default = todos en 1.0.

`weights(self) -> torch.Tensor` `softplus(raw_w) -> tensor [wEE, wEI, wIE, wII]` positivo, **con gradiente**.

`weights_dict(self) -> dict[str, float]` Los mismos pesos como floats (`.detach()`), para loguear.

Clase MultiTrajectoryTrainer

Entrena las N redes (warmup) y luego el θ compartido (congelando las redes).

__init__(self, model: MultiTrajPINN, config, fixed_params: dict) Manda el modelo al device. Optimizador **Adam** con dos grupos: todas las redes (`model.nets.parameters()`) con `lr`, y el θ compartido (`raw_w`) con `lr_weights`. Mismo `ReduceLROnPlateau` que el Trainer de una trayectoria. Usa la misma clase `TrainConfig` de `train.py`.

_params(self) -> dict Junta `self.fixed` con el θ compartido: arma un dict con los fijos y sobrescribe `wEE`, `wEI`, `wIE`, `wII` con `model.weights()` (tensores). Es lo que reciben las `physics_loss` de cada trayectoria.

_prepare(self, datasets: list[dict]) -> list[dict] Convierte cada dataset a tensores columna y arma una lista de dicts, uno por trayectoria, con `t`, `target`, `P`, `Q`, `t0`, `ic`, `n`.

train(self, datasets: list[dict]) -> dict Bucle en **dos fases**, controladas por `physics_warmup`:

- **Transición** (`epoch == warmup`): resetea `lr` de los dos grupos, reinstancia el scheduler, reinicia la EMA, y **congela** todas las redes (`requires_grad_(False)` sobre `model.nets.parameters()`).
- **Fase 1 (warmup, epoch < warmup)**: cada red aprende su trayectoria. Suma sobre las N trayectorias L_d (minibatch de datos, tamaño `batch_size`) + L_{ic} . $L = w_{data}L_d + w_{ic}L_{ic}$. La física está apagada.
- **Fase 2 (epoch >= warmup)**: con las redes congeladas, ajusta solo θ . $L_f = \text{suma de physics_loss}$ sobre las N trayectorias (cada una con sus `n_collocation` puntos y sus propios `P`, `Q`). $L = w_{physics}L_f$.
- Registro, EMA, `scheduler.step`, log cada `log_every`, y **detección de meseta** idéntica al Trainer (solo tras el warmup).
- Devuelve `hist` con `loss`, `data`, `physics`, una lista por peso, y `stop_epoch`, `warmup`.

Notas / diferencias con el Trainer de una trayectoria: - No hace validación temporal (`val_fraction` / `_val_mse` / corte train-test): usa **todos** los puntos de cada trayectoria tanto para datos (fase 1) como para colocación (fase 2). - No tiene `save_checkpoint` / `load_checkpoint`.

predict(self, i: int, t_array) -> np.ndarray (@torch.no_grad) Predicción de la red i -ésima sobre tiempos dados; numpy (N , 2).

Notas finales

- **No hay stubs** en este paquete: los cuatro archivos de lógica (`network`, `losses`, `train`, `multitraj`) están completos y operativos.
- El acoplamiento clave entre módulos es `physics_loss`: acepta parámetros como float o como tensor, y ese único detalle permite reusar exactamente el mismo residuo para el forward (Paso 4.0), el inverso de una trayectoria (Trainer) y el inverso multi-trayectoria (MultiTrajectoryTrainer).
- La estructura del residuo (umbral adentro de la entrada, offset k_e/k_i restado) debe coincidir **exactamente** con el rhs del simulador (`model.py`); ver **P0**. Si el simulador cambiara la forma de la ecuación, hay que actualizar `physics_loss` en espejo.
- Comparación PINN vs Neural ODE: ver la figura `pinns_vs_node.png` y **P4 (neural_ode)**.